

Chapter 1: Introduction to Semantic Search

Semantic search is an information retrieval approach that goes beyond simple keyword matching. Instead of looking for exact word overlaps, it uses vector embeddings to capture the underlying meaning of a query and the documents being searched.

This allows a search system to return relevant results even when the words used in the query do not exactly match the words in the source text. For example, a search for 'car repair shop' could return documents about 'automobile service centers' because their meanings are close in vector space.

The foundation of semantic search is the embedding model, which converts text into a high dimensional numeric vector. Texts with similar meaning end up close together in this vector space, while unrelated texts are placed far apart.

Chapter 2: Embeddings and Vector Representations

An embedding is a dense numerical representation of text, typically a list of floating point numbers ranging from a few hundred to a few thousand dimensions. Modern embedding models, such as Google's gemini-embedding-001, are trained on massive text corpora to learn these representations.

Once text is converted into embeddings, similarity between two pieces of text can be measured using distance metrics such as cosine similarity or Euclidean distance. A smaller distance indicates greater semantic similarity.

Embeddings are not limited to single words; entire sentences, paragraphs, or documents can be embedded as a single vector, which is especially useful for retrieval augmented generation systems that need to compare a user's question against chunks of a larger document.

Chapter 3: Chunking Strategies

Before documents can be embedded, they are usually split into smaller pieces called chunks. This is necessary because embedding models have input length limits, and smaller chunks tend to produce more focused and accurate embeddings.

A common strategy is the `RecursiveCharacterTextSplitter`, which tries to split text along natural boundaries such as paragraphs and sentences before falling back to splitting by raw character count. Overlap between chunks, typically 100 to 200 characters, helps preserve context that might otherwise be lost at chunk boundaries.

Choosing the right chunk size is a balance: chunks that are too large may dilute the specific meaning being searched for, while chunks that are too small may lose important surrounding context.

Chapter 4: Vector Stores and Retrieval

A vector store is a specialized database designed to store embeddings and perform fast similarity search across potentially millions of vectors. FAISS, developed by Meta, is a popular open source library for this purpose and is widely used in local or self-hosted retrieval augmented generation pipelines.

When a user asks a question, the question itself is embedded using the same embedding model used for the documents. The vector store then finds the most similar document chunks, typically returning the top-k results, where k might be 3, 4, or 5 depending on the application.

These retrieved chunks are then passed as context to a large language model, which uses them to generate a grounded, accurate answer rather than relying purely on its own internal knowledge.

Chapter 5: Building a Complete RAG Pipeline

A retrieval augmented generation, or RAG, pipeline typically consists of several stages: loading source documents, splitting them into chunks, embedding those chunks, storing them in a vector database, retrieving relevant chunks for a given query, and finally generating an answer using a language model.

Frameworks like LangChain provide composable building blocks, such as document loaders, text splitters, retrievers, and output parsers, that can be chained together using the LangChain Expression Language, often written with the pipe operator.

A well designed RAG system reduces hallucination by grounding the model's answers in retrieved source material, and it allows the underlying knowledge base to be updated without needing to retrain the language model itself.